Open in Colab

Warmup: fundamentals

Recall that last week's objectives were:

- Describe fundamental aspects of a working Python script
- Implement novel functions and for loops along with appropriate unit tests
- Utilize 'external' python packages

This warm-up aims to build on those objectives and reinforce them.

1. Basics of functions

1(a): ins and outs

Run the following cell. It doesn't work as expected. Explain why (in a markdown cell below it) and create a new cell below that with a fixed version of the code.

In [2]:
```
def factorial(n):
    """Calculates the factorial of a number."""
    fac = 1
    if n == 1:
        fac = 1
    else:
        fac = n * factorial(n-1)
    return fac

# test the factorial function; this should print 120
print(factorial(5))
```

```
120
```

120

In [3]:
```
def factorial(n):
    fac = 1
    if n == 1:
        fac = 1
    else:
        fac = n * factorial(n-1)
    return fac
print(factorial(5))
```

```
120
```

1(b): Writing new functions

Write a function representing the formula $z = x^2 + y^2$ and verify that it works as expected.

In [4]:
```
def calculate_z(x, y):
    z = x**2 + y**2
    return z

print(calculate_z(2, 3))
```

```
13
```

1(c): Scope

We haven't yet discussed the concept of *scope*, so we'll introduce it here through examining an interesting effect (and we'll discuss it more as time goes on.)

Run the cell below. We see that the value of `a` is redefined when `redefine_and_print_a()` is called, so why does the last print statement contain the original definition of `a` and not the one it was given within the function?

In [5]:
```
# define a as a global variable here
a = "This is `a` with global scope."
# print a to show its value
print(f"1: {a}")
# define a function
def redefine_and_print_a():
    # define a as a local variable here
    a = "This is `a` with local scope."
    # print a to show its value
    print(f"2: {a}")

# call the function
redefine_and_print_a()

# print a one more time to show its value
print(f"3: {a}")
```

```
1: This is `a` with global scope.
2: This is `a` with local scope.
3: This is `a` with global scope.
```

1: This is `a` with global scope. 2: This is `a` with local scope. 3: This is `a` with global scope.

2. Using external packages

Make a plot of the following equation using `numpy` and `matplotlib`. Use `x` values ranging from -8 to 8, and make plots using the following values of σ: $\sigma = [0.25,0.5,1,2,4]$

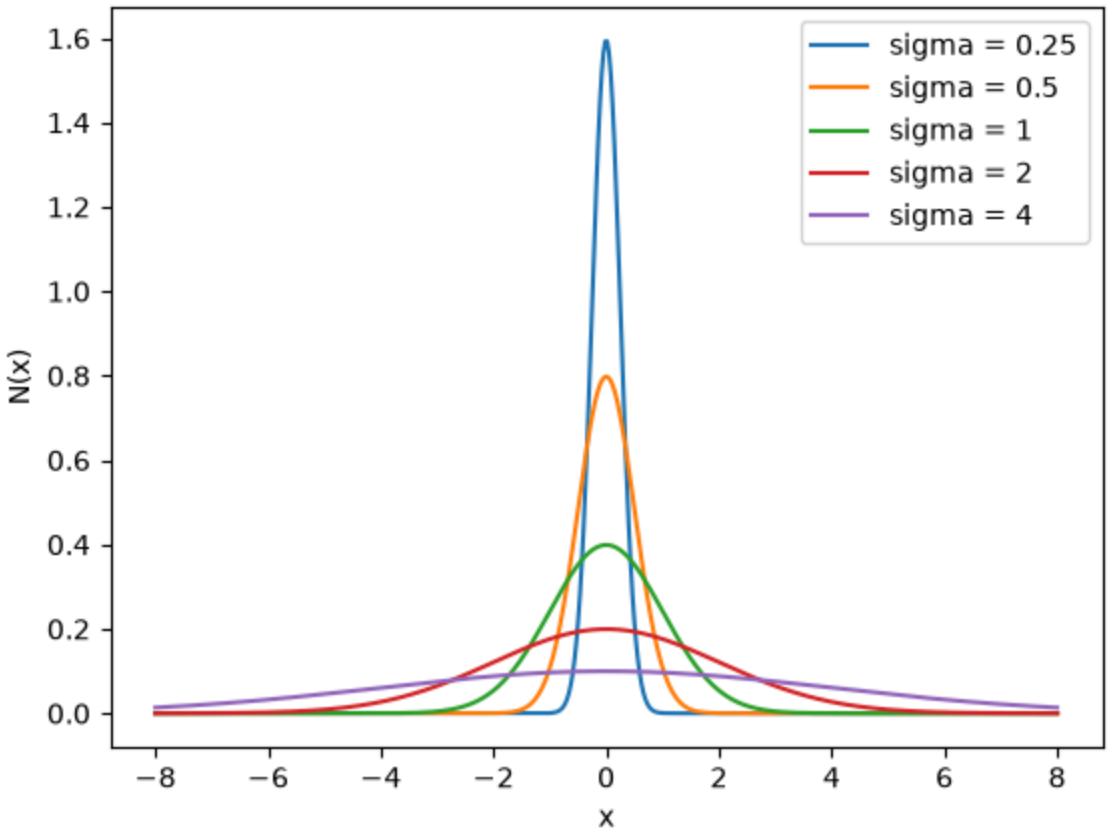
$$N(x) = \frac{1}{\sigma \sqrt{2 \pi}} e^{-\frac{x^2}{2 \sigma^2}}$$

Your solution should (a) define a new function to represent this formula, and (b) use a `for` loop.

If you have time, customize the x- and y-axis labels, and add a legend.

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
```

In [3]:
```
def N(x, sigma):
    return (1 / (sigma * np.sqrt(2 * np.pi))) * np.exp(-x**2 / (2 * sigma**2
x = np.linspace(-8, 8, 500)
sigmas = [0.25, 0.5, 1, 2, 4]
for sigma in sigmas:
    plt.plot(x, N(x, sigma), label=f"sigma = {sigma}")
plt.xlabel("x")
plt.ylabel("N(x)")
plt.legend()
plt.show()
```



In [ ]:
```
```